

Material de Patrones de Diseño GoF para la Técnica de Aronson

Prof. Daniel San Martín

17 de abril de 2025

1. Patrones de Comportamiento

El diseñador de un sistema orientado a objetos se enfrenta a menudo al problema del descubrimiento de objetos. Esto puede realizarse a partir de los dos aspectos siguientes:

- La estructuración de los datos.
- La distribución de los procesamiento y de los algoritmos.

Los patrones de estructuración aportan soluciones a los problemas de estructuración de datos y de objetos.

El objetivo de los patrones de comportamiento consiste en proporcionar soluciones para distribuir el procesamiento y los algoritmos entre los objetos.

Estos patrones organizan los objetos así como sus interacciones especificando los flujos de control y de procesamiento en el seno de un sistema de objetos.

1.1. Observer

1.1.1. Descripción

El patrón Observer tiene como objetivo construir una dependencia entre un sujeto y los observadores de modo que cada modificación del sujeto sea notificada a los observadores para que puedan actualizar su estado.

1.1.2. Ejemplo

Queremos actualizar la visualización de un catálogo en tiempo real. Cada vez que la información relativa a un vehículo se modifica, queremos actualizar la visualización de la misma. Puede haber varias visualizaciones simultáneas.

La solución recomendada por el patrón Observer consiste en establecer un enlace entre cada vehículo y sus vistas para que el vehículo pueda indicarles que se actualicen cuando su estado interno haya sido modificado. Esta solución se ilustra en la Figura 1.

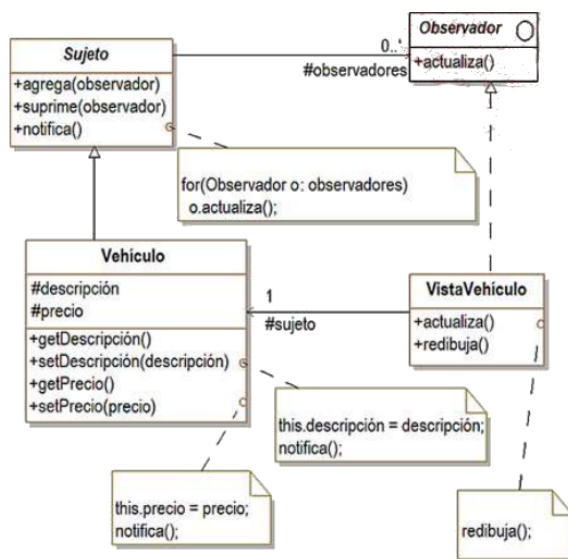


Figura 1: El patrón Observer aplicado a la visualización de vehículos

El diagrama contiene las cuatro clases siguientes:

- **Sujeto** es la clase abstracta que incluye todo objeto que notifica a los demás objetos de las modificaciones en su estado interno.
- **Vehículo** es la subclase concreta de Sujeto que describe a los vehículos. Gestiona dos atributos: descripción y precio.
- **Observador** es la interfaz de todo objeto que necesite recibir las notificaciones de cambio de estado provenientes de los objetos a los que se ha inscrito previamente.
- **VistaVehículo** es la subclase concreta correspondiente a la implementación de Observador cuyas instancias muestran la información de un vehículo.

El funcionamiento es el siguiente: cada nueva vista se inscribe como observador de su vehículo mediante el método `agrega`. Cada vez que la descripción o el precio se actualizan, se invoca el método `notifica`. Éste solicita a todos los observadores que se actualicen, invocando a su método `actualiza`. En la clase `VistaVehículo`, éste último método se llama `redibuja`. Este funcionamiento se ilustra en la Figura 2 mediante un diagrama de secuencia.

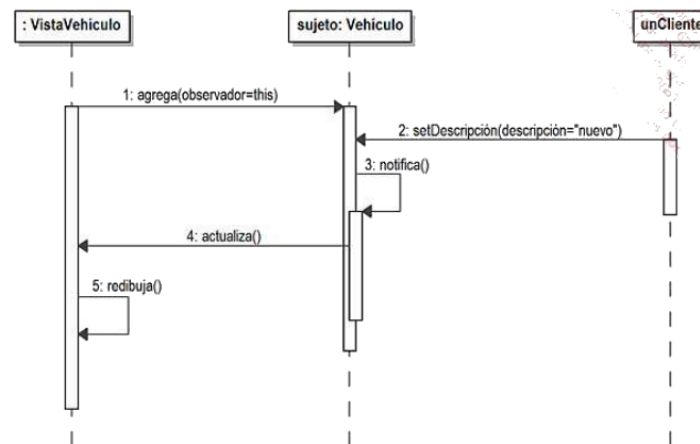


Figura 2: Diagrama de secuencia detallando el uso del patrón Observer

La solución implementada mediante el patrón Observer es genérica. En efecto, todo el mecanismo de observación está implementado en la clase `Sujeto` y en la interfaz `Observador`, que puede tener otras subclases distintas de `Vehículo` y `VistaVehículo`.

1.2. Estructura

La Figura 3 detalla la estructura genérica del patrón.

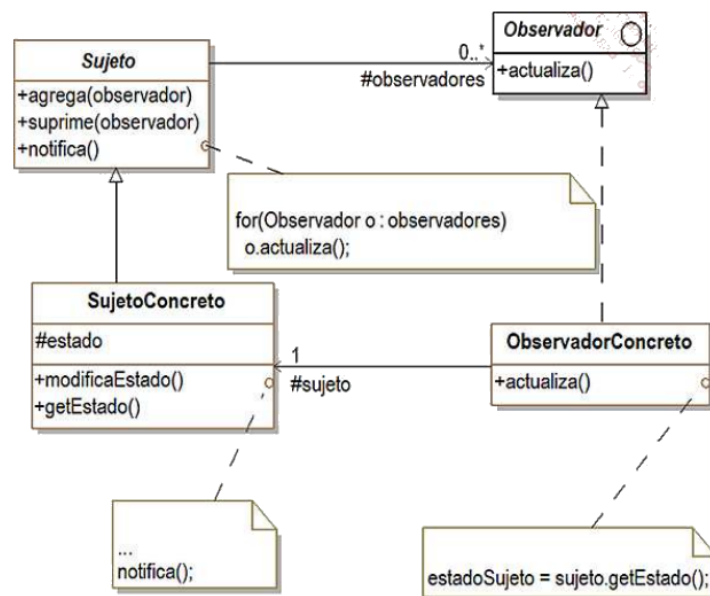


Figura 3: Estructura del patrón Observer

1.2.1. Participantes

Los participantes del patrón son los siguientes:

- **Sujeto** es la clase abstracta que incluye la asociación con los observadores así como los métodos para agregar o suprimir observadores.
- **Observador** es la interfaz que es necesario implementar para recibir las notificaciones (método actualiza).
- **SujetoConcreto** (Vehículo) es una clase correspondiente a la implementación de un sujeto. Un sujeto envía una notificación cuando su estado se ha modificado.
- **ObservadorConcreto** (VistaVehículo) es una clase de implementación de un observador. Mantiene una referencia hacia el sujeto e implementa el método actualiza. Solicita a su sujeto información que forma parte de su estado durante las actualizaciones invocando al método getEstado.

1.2.2. Colaboraciones

El sujeto notifica a sus observadores cuando su estado interno ha sido modificado. Cuando un observador recibe esta notificación, se actualiza en consecuencia. Para realizar esta actualización, puede invocar a los métodos del sujeto que dan acceso a su estado.

1.3. Dominios de Aplicación

El patrón se utiliza en los casos siguientes:

- Una modificación en el estado de un objeto genera modificaciones en otros objetos que se determinan dinámicamente.
- Un objeto quiere avisar a otros objetos sin tener que conocer su tipo, es decir sin estar fuertemente acoplado a ellos.
- No se desea fusionar dos objetos en uno solo.

Retomamos el ejemplo de la Figura 2. El código fuente de la clase Sujeto aparece a continuación. Los observadores se gestionan mediante una lista.

```
1 import java.util.*;
2 public abstract class Sujeto {
3
4     protected List<Observador> observadores = new ArrayList<Observador>();
5
6     public void agrega(Observador observador) {
7         observadores.add(observador);
8     }
9
10    public void suprime(Observador observador) {
11        observadores.remove(observador);
12    }
13
14    public void notifica() {
15        for (Observador observador: observadores)
16            observador.actualiza();
17    }
18 }
```

El código fuente de la interfaz Observador es muy simple puesto que sólo contiene la firma del método actualiza.

```
1 public interface Observador {
2     void actualiza();
3 }
```

El código fuente de la clase Vehículo aparece a continuación. Contiene dos atributos y los accesos de lectura y escritura para ambos atributos. Los dos accesos de escritura invocan al método notifica.

```
1 public class Vehiculo extends Sujeto {
2     protected String descripcion;
3
4     protected Double precio;
5     public String getDescripcion() {
6         return descripcion;
7     }
8     public void setDescripcion(String descripcion) {
9         this.descripcion = descripcion;
10    }
```

```

10     this.notifica();
11 }
12 public Double getPrecio() {
13     return precio;
14 }
15 public void setPrecio(Double precio) {
16     this.precio = precio;
17     this.notifica();
18 }
19 }

```

La clase `VistaVehiculo` gestiona un texto que contiene la descripción y el precio del vehículo asociado (el sujeto). Este texto se actualiza tras cada notificación en el cuerpo del método `actualiza`. El método `redibuja` imprime este texto por pantalla.

```

1 public class VistaVehiculo implements Observador {
2
3     protected Vehiculo vehiculo;
4     protected String texto = "";
5
6     public VistaVehiculo(Vehiculo vehiculo) {
7         this.vehiculo = vehiculo;
8         vehiculo.agrega(this);
9         actualizaTexto();
10    }
11    protected void actualizaTexto() {
12        texto = "Descripcion " + vehiculo.descripcion + " Precio: " + vehiculo.precio;
13    }
14    public void actualiza() {
15        actualizaTexto();
16        this.redibuja();
17    }
18
19    public void redibuja() {
20        System.out.println(texto);
21    }
22 }

```

Por último, la clase `Usuario` incluye el programa principal. Este programa crea un vehículo y a continuación una vista a la que le pide la visualización. A continuación se modifica el precio y la vista se refresca. A continuación se crea una segunda vista que se asocia al mismo vehículo. El precio se modifica de nuevo y ambas vistas se refrescan.

```

1 public class Usuario {
2     public static void main(String[] args) {
3         Vehiculo vehiculo = new Vehiculo();
4         vehiculo.setDescripcion("Vehiculo economico");
5         vehiculo.setPrecio(5000.0);
6         VistaVehiculo vistaVehiculo = new VistaVehiculo(vehiculo);
7         vistaVehiculo.redibuja();
8         vehiculo.setPrecio(4500.0);
9         VistaVehiculo vistaVehiculo2 = new VistaVehiculo(vehiculo);
10        vehiculo.setPrecio(5500.0);
11    }
12 }

```

El resultado de la ejecución de este programa es el siguiente.

```

Descripcion vehiculo economico precio: 5000.0
Descripcion vehiculo economico precio: 4500.0
Descripcion vehiculo economico precio: 5500.0
Descripcion vehiculo economico precio: 5500.0

```

1.4. Strategy

1.4.1. Descripción

El patrón Strategy tiene como objetivo adaptar el comportamiento y los algoritmos de un objeto en función de una necesidad sin cambiar las interacciones de este objeto con los clientes.

Esta necesidad puede ponerse de relieve en base a aspectos tales como la presentación, la eficacia en tiempo de ejecución o en memoria, la elección de algoritmos, la representación interna, etc. Aunque evidentemente no se trata de una necesidad funcional de cara a los clientes del objeto, pues las interacciones entre el objeto y sus clientes deben permanecer inmutables.

1.4.2. Ejemplo

En el sistema de venta online de vehículos, la clase **VistaCatálogo** dibuja la lista de vehículos destinados a la venta. Se utiliza un algoritmo de diseño gráfico para calcular la representación gráfica en función del navegador. Existen dos versiones de este algoritmo:

- Una primera versión que sólo muestra un vehículo por línea (un vehículo ocupa todo el ancho disponible) y que muestra toda la información posible así como cuatro fotografías.
- Una segunda versión que muestra tres vehículos por línea pero que muestra menos información y una única fotografía.

La interfaz de la clase **VistaCatálogo** no depende de la elección del algoritmo de representación gráfica. Esta elección no tiene impacto alguno en la relación de una vista de catálogo con sus clientes. Sólo se modifica la representación.

Una primera solución consiste en transformar la clase **VistaCatálogo** en una interfaz o en una clase abstracta y en incluir dos subclases de implementación diferentes según la elección del algoritmo. Esto presenta el inconveniente de complicar de manera inútil la jerarquía de las vistas del catálogo.

Otra posibilidad consiste en implementar ambos algoritmos en la clase **VistaCatálogo** y en apoyarse en instrucciones condicionales para realizar la elección. No obstante esto consiste en desarrollar una clase relativamente pesada donde el código de los métodos es difícil de comprender.

El patrón Strategy proporciona otra solución incluyendo una clase por algoritmo. El conjunto de las clases así creadas posee una interfaz común que se utiliza para dialogar con la clase **VistaCatálogo**. La figura 4 muestra el diagrama de clases de la aplicación del patrón Strategy.

Este diagrama muestra las dos clases de algoritmos: **DibujaUnVehículoPorLínea** y **DibujaTresVehículosPorLínea** que implementan la interfaz **DibujaCatálogo**. La nota que detalla el método **dibuja** de la clase **VistaCatálogo** muestra cómo se utilizan ambos métodos.

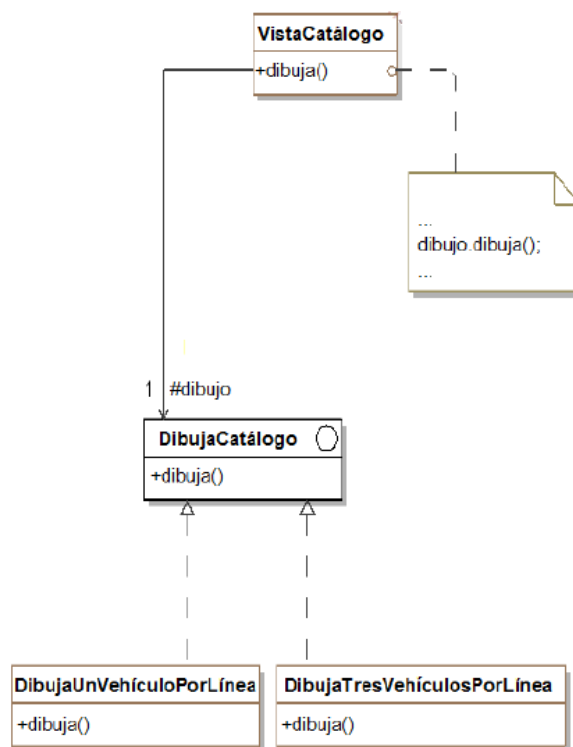


Figura 4: Aplicación del patrón Strategy para dibujar un catálogo de vehículos

1.4.3. Estructura

La Figura 5 muestra la estructura genérica del patrón.

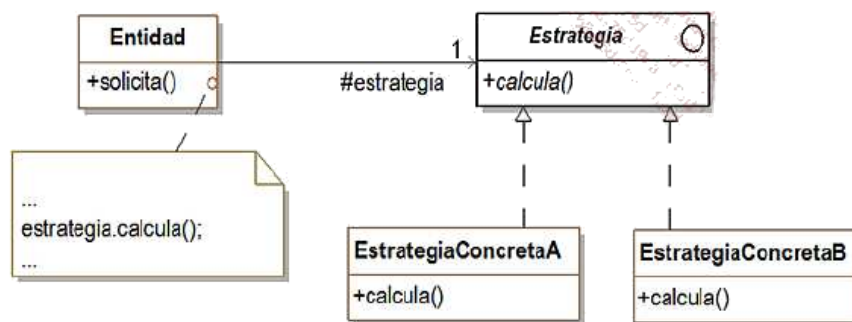


Figura 5: Estructura del patrón Strategy

1.4.4. Participantes

Los participantes del patrón son los siguientes:

- **Estrategia** (DibujaCatálogo) es la interfaz común a todos los algoritmos. Esta interfaz se utiliza en **Entidad** para invocar al algoritmo.
- **EstrategiaConcretaA** y **EstrategiaConcretaB** (DibujaUnVehículoPorLínea y DibujaTresVehículosPorLínea) son las subclases concretas que implementan los distintos algoritmos.
- **Entidad** es la clase que utiliza uno de los algoritmos de las clases que implementan la **Estrategia**. Por consiguiente, posee una referencia hacia una de estas clases. Por último, si fuera necesario, puede exponer sus datos internos a las clases de implementación.

1.4.5. Colaboraciones

La entidad y las instancias de las clases de implementación de la **Estrategia** interactúan para implementar los algoritmos. En el caso más sencillo, los datos que necesita el algoritmo se pasan como parámetro. Si fuera necesario, la clase **Entidad** implementaría los métodos necesarios para dar acceso a sus datos internos.

El cliente inicializa la entidad con una instancia de la clase de implementación de **Estrategia**. Él mismo selecciona esta clase y, por lo general, no la modifica a continuación. La entidad puede modificar a continuación esta elección.

La entidad redirige las peticiones provenientes de sus clientes hacia la instancia referenciada por su atributo **estrategia**.

1.4.6. Dominios de Aplicación

El patrón se utiliza en los casos siguientes:

- El comportamiento de una clase puede estar implementado mediante distintos algoritmos siendo alguno de ellos más eficaz en términos de ejecución o de consumo de memoria o incluso contener mecanismos de decisión.
- La implementación de la elección del algoritmo mediante instrucciones condicionales se vuelve demasiado compleja.
- Un sistema posee numerosas clases idénticas salvo una parte correspondiente a su comportamiento.

1.4.7. Ejemplo en Java

Nuestro ejemplo escrito en Java está basado en la visualización del catálogo de vehículos, simulado aquí simplemente mediante salidas por pantalla.

La interfaz **DibujaCatalogo** incluye el método **dibuja** que recibe como parámetro una lista de instancias de **VistaVehiculo**.

```
1 import java.util.*;
2 public interface DibujaCatalogo {
3     void dibuja(List<VistaVehiculo> contenido);
4 }
```

La clase **DibujaUnVehiculoPorLinea** implementa el método **dibuja** mostrando un vehículo por cada línea (imprime un salto de línea tras mostrar un vehículo).

```

1 import java.util.*;
2 public class DibujaUnVehiculoPorLinea implements DibujaCatalogo {
3
4     public void dibuja(List<VistaVehiculo> contenido) {
5         System.out.println( "Dibuja los vehiculos mostrando un vehiculo por linea");
6         for (VistaVehiculo vistaVehiculo: contenido) {
7             vistaVehiculo.dibuja();
8             System.out.println();
9         }
10
11         System.out.println();
12     }
13 }

```

La clase DibujaTresVehiculosPorLinea implementa el método dibuja mostrando tres vehículos por línea (imprime un salto de línea tras mostrar tres vehículos).

```

1 import java.util.*;
2 public class DibujaTresVehiculosPorLinea implements DibujaCatalogo {
3
4     public void dibuja(List<VistaVehiculo> contenido) {
5         int contador;
6         System.out.println("Dibuja los vehiculos mostrando tres vehiculos por linea");
7         contador = 0;
8         for (VistaVehiculo vistaVehiculo: contenido) {
9             vistaVehiculo.dibuja();
10            contador++;
11            if (contador == 3) {
12                System.out.println();
13                contador = 0;
14            }
15            else
16                System.out.println(" ");
17        }
18        if (contador != 0)
19            System.out.println();
20        System.out.println();
21    }
22 }

```

La clase VistaVehiculo que dibuja un vehículo tiene el código que se muestra a continuación. Conviene observar que el método dibuja de VistaVehiculo, en el caso de esta simulación, es idéntico para la visualización en una línea que en tres líneas, lo cual no suele ser el caso en la realidad de una interfaz gráfica.

```

1 public class VistaVehiculo {
2
3     protected String descripcion;
4
5     public VistaVehiculo(String descripcion) {
6         this.descripcion = descripcion;
7     }
8
9     public void dibuja() {
10        System.out.print(descripcion);
11    }
12 }

```

La clase VistaCatalogo posee un constructor que recibe como parámetro una instancia de una de las clases de implementación de DibujaCatalogo, instancia que se memoriza en el atributo dibujo. Este constructor inicializa a su vez el contenido que normalmente debería leerse desde una base de datos.

El método dibuja redirige la llamada hacia la instancia memorizada en dibujo. Pasa como parámetro una referencia hacia el contenido del catálogo.

```

1 import java.util.*;
2 public class VistaCatalogo {
3
4     protected List<VistaVehiculo> contenido = new ArrayList<VistaVehiculo>();
5     protected DibujaCatalogo dibujo;
6
7     public VistaCatalogo(DibujaCatalogo dibujo) {
8         contenido.add(new VistaVehiculo("vehiculo economico"));
9         contenido.add(new VistaVehiculo("vehiculo amplio"));
10        contenido.add(new VistaVehiculo("vehiculo rapido"));
11        contenido.add(new VistaVehiculo("vehiculo confortable"));

```

```

12         contenido.add(new VistaVehiculo("vehiculo deportivo"));
13         this.dibujo = dibujo;
14     }
15
16     public void dibuja() {
17         dibujo.dibuja(contenido);
18     }
19 }

```

Por último, el programa principal está implementado mediante la clase Usuario. Éste crea dos instancias de VistaCatalogo, la primera está configurada para realizar un dibujo en tres líneas. La segunda instancia está configurada para realizar un dibujo en una línea. Tras haberlas creado, el programa principal invoca al método dibuja de sus instancias.

```

1 public class Usuario {
2
3     public static void main(String[] args) {
4
5         VistaCatalogo vistaCatalogo1 = new VistaCatalogo(new DibujaTresVehiculosPorLinea());
6         vistaCatalogo1.dibuja();
7         VistaCatalogo vistaCatalogo2 = new VistaCatalogo(new DibujaUnVehiculoPorLinea());
8         vistaCatalogo2.dibuja();
9     }
10 }

```

La ejecución de este programa produce el resultado siguiente, que muestra claramente cómo el comportamiento del método dibuja está configurado por la instancia que se pasa como parámetro al constructor.

```

Dibuja los vehiculos mostrando tres vehiculos por linea
vehiculo economico vehiculo amplio vehiculo rapido
vehiculo confortable vehiculo deportivo
Dibuja los vehiculos mostrando un vehiculo por linea
vehiculo economico
vehiculo amplio
vehiculo rapido
vehiculo confortable
vehiculo deportivo

```


1.5. Template Method

1.5.1. Descripción

El patrón Template Method permite delegar en las subclases ciertas etapas de una de las operaciones de un objeto, estando estas etapas descritas en las subclases.

1.5.2. Ejemplo

En el sistema de venta online de vehículos, queremos gestionar pedidos de clientes de España y de Luxemburgo. La diferencia entre ambas peticiones concierne principalmente al cálculo del IVA. Si bien en Chile la tasa de IVA es siempre fija del 21 %, en el caso de Luxemburgo es variable (12 % para los servicios, 15 % para el material). El cálculo del IVA requiere dos operaciones de cálculo distintas en función del país.

Una primera solución consiste en implementar dos clases distintas sin súperclase común: `PedidoEspaña` y `PedidoLuxemburgo`. Esta solución presenta el inconveniente importante de que tiene código idéntico que no ha sido factorizado, como por ejemplo la visualización de la información del pedido (método `visualiza`).

Podría incluirse una clase abstracta `Pedido` para factorizar los métodos comunes, como el método `visualiza`. El patrón Template Method permite ir más lejos proponiendo factorizar el código común en el interior de los métodos. Tomemos el ejemplo del método `calculaImporteConIVA` cuyo algoritmo es el siguiente para España (escrito en pseudo-código).

```
calculaImporteConIVA:
importeIVA = importeSinIVA * 0,21;
importeConIVA = importeSinIVA + importeIVA;
```

El algoritmo para Luxemburgo tiene el siguiente pseudo-código.

```
calculaImporteConIVA:
importeIVA = (importeServiciosSinIVA * 0,12) + (importeMaterialSinIVA * 0,15);
importeConIVA = importeSinIVA + importeIVA;
```

Vemos en este ejemplo que la última línea del método es común a ambos países (en este ejemplo, sólo hay una línea común aunque en un caso real la parte común puede ser mucho más importante).

Reemplazamos la primera línea por una llamada a un nuevo método llamado `calculaIVA`. De este modo el método `calculaImporteConIVA` se describe en adelante de la siguiente forma:

```
calculaImporteConIVA:
calculaIVA();
importeConIVA = importeSinIVA + importeIVA;
```

El método `calculaImporteConIVA` puede ahora factorizarse. El código específico ha sido desplazado en el método `calculaIVA`, cuya implementación es específica para cada país. El método `calculaIVA` se incluye en la clase `Pedido` como método abstracto.

El método `calculaImporteConIVA` se llama un método “modelo” (template method). Un método “modelo” incluye la parte común de un algoritmo que está complementado por partes específicas.

Esta solución se ilustra en el diagrama de clases de la Figura 6 donde, por motivos de simplicidad, el cálculo del IVA de Luxemburgo se ha configurado con una tasa única del 15 %.

Cuando un cliente invoca al método `calculaImporteConIVA` de un pedido, éste invoca al método `calculaIVA`.

La implementación de este método depende de la clase concreta del pedido:

- Si esta clase es `PedidoEspaña`, el diagrama de secuencia se describe en la Figura 7. El IVA se calcula basado en la tasa del 21 %.
- Si esta clase es `PedidoLuxemburgo`, el diagrama de secuencia se describe en la Figura 8. El IVA se calcula basado en la tasa del 15 %.

1.5.3. Estructura

La Figura 9 muestra la estructura genérica del patrón.

1.5.4. Participantes

Los participantes del patrón son los siguientes:

- La clase abstracta `ClaseAbstracta` (`Pedido`) incluye el método “modelo” así como la firma de los métodos abstractos que invoca este método.
- La subclase concreta `ClaseConcreta` (`PedidoEspaña` y `PedidoLuxemburgo`) implementa los métodos abstractos utilizados por el método “modelo” de la clase abstracta. Puede haber varias clases concretas.

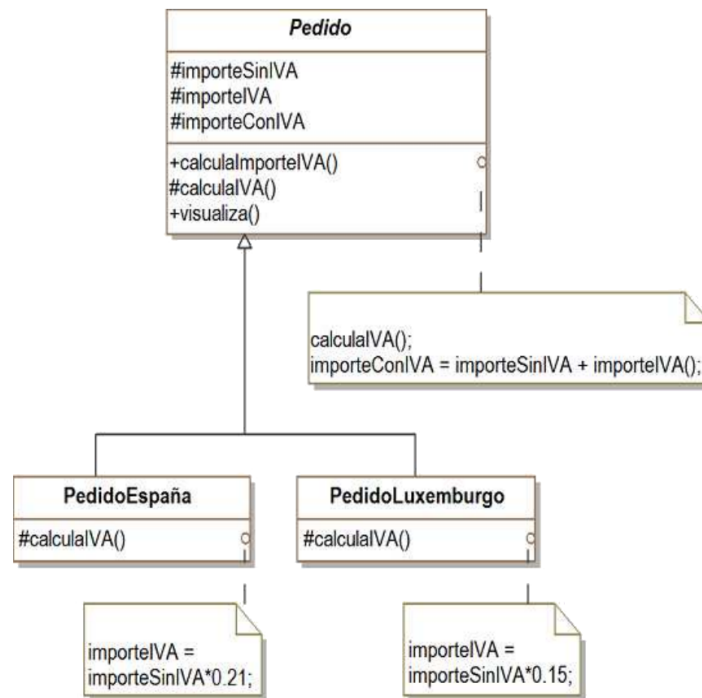


Figura 6: Aplicación del patrón Template Method para el cálculo del IVA de un pedido en función del país

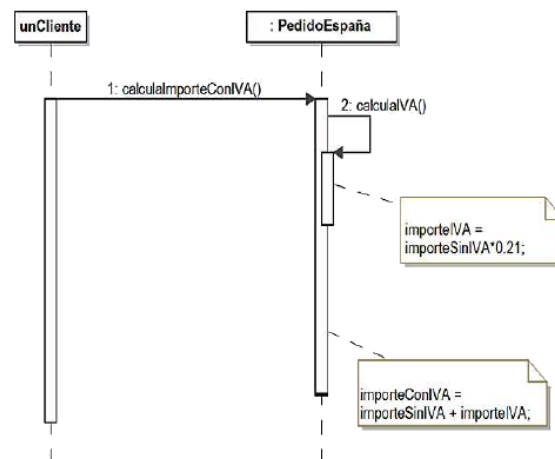


Figura 7: Diagrama de secuencia correspondiente al cálculo del importe con IVA de un pedido español

1.5.5. Colaboraciones

La implementación del algoritmo se realiza mediante la colaboración entre el método “modelo” de la clase abstracta y los métodos de una subclase concreta que complementa el algoritmo.

1.5.6. Dominios de Aplicación

El patrón se utiliza en los casos siguientes:

- Una clase compartida con otra u otras clases con código idéntico que puede factorizarse siempre que las partes específicas a cada clase hayan sido desplazadas a nuevos métodos.
- Un algoritmo a posee una parte invariante y partes específicas a distintos tipos de objetos.

1.5.7. Ejemplos en Java

La clase abstracta `Pedido` incluye el método “modelo” `calculaImporteConIVA` que invoca al método abstracto `calculaIVA`.

```

1 public abstract class Pedido {
2     protected double importeSinIVA;

```

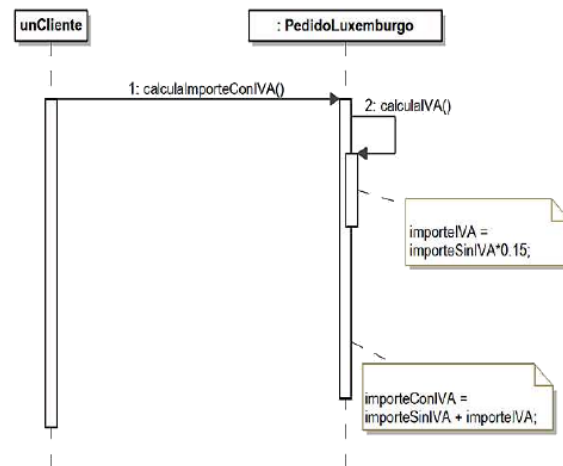


Figura 8: Diagrama de secuencia correspondiente al cálculo del importe con IVA de un pedido español

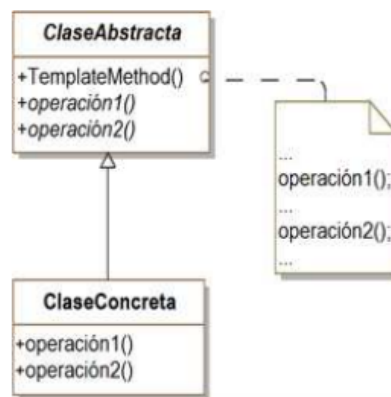


Figura 9: Estructura del patrón Template Method

```

3  protected double importeIVA;
4  protected double importeConIVA;
5  protected abstract void calculaIVA();
6  public void calculaPrecioConIVA() {
7      this.calculaIVA();
8      importeConIVA = importeSinIVA + importeIVA;
9  }
10 public void setImporteSinIVA(double importeSinIVA) {
11     this.importeSinIVA = importeSinIVA;
12 }
13
14 public void visualiza() {
15     System.out.println("Pedido");
16     System.out.println("Importe sin IVA " + importeSinIVA);
17     System.out.println("Importe con IVA " + importeConIVA);
18 }
19 }
  
```

La subclase concreta PedidoEspana implementa el método calculaIVA con la tasa de IVA español.

```

1  public class PedidoEspanha extends Pedido {
2      protected void calculaIVA() {
3          importeIVA = importeSinIVA * 0.21;
4      }
5  }
  
```

La subclase concreta PedidoLuxemburgo implementa el método calculaIVA con la tasa de IVA luxemburgués.

```

1  public class PedidoLuxemburgo extends Pedido {
2      protected void calculaIVA() {
3          importeIVA = importeSinIVA * 0.15;
4      }
5  }
  
```

Por último, la clase Usuario contiene el programa principal. Éste crea un pedido español, fija el importe sin IVA, calcula el importe con IVA y a continuación muestra el pedido. A continuación, el programa principal realiza la misma operación con un pedido luxemburgués.

```
1 public class Usuario {
2
3     public static void main(String[] args) {
4
5         Pedido pedidoEspanha = new PedidoEspanhaa();
6         pedidoEspanha.setImporteSinIVA(10000);
7         pedidoEspanha.calculaPrecioConIVA();
8         pedidoEspanha.visualiza();
9
10        Pedido pedidoLuxemburgo = new PedidoLuxemburgo();
11        pedidoLuxemburgo.setImporteSinIVA(10000);
12        pedidoLuxemburgo.calculaPrecioConIVA();
13        pedidoLuxemburgo.visualiza();
14    }
15 }
```

La ejecución del programa produce el siguiente resultado.

```
Pedido
Importe sin IVA 10000.0
Importe con IVA 12100.0
Pedido
Importe sin IVA 10000.0
Importe con IVA 11500.0
```

1.6. Visitor

1.6.1. Descripción

El patrón Visitor construye una operación que debe realizarse sobre los elementos de un conjunto de objetos. Esto permite agregar nuevas operaciones sin modificar las clases de estos objetos.

1.6.2. Ejemplo

Consideremos la Figura 10 que describe los clientes de nuestro sistema organizados bajo la forma de objetos compuestos según el patrón **Composite**. A excepción del método **agregaFilial**, específico a la gestión de la composición, las dos subclases poseen dos métodos con el mismo nombre: **calculaCosteEmpresa** y **envíaEmailComercial**. Cada uno de estos métodos se corresponde con una funcionalidad cuya implementación está bien adaptada en función de la clase. Podrían implementarse muchas otras funcionalidades como, por ejemplo, el cálculo de la cifra de negocios de un cliente (incluyendo o no sus filiales), etc.

En el diagrama, el cálculo del costo del mantenimiento no está detallado. Este enfoque puede utilizarse siempre y cuando el número de funcionalidades sea pequeño. En cambio, si se vuelve importante, obtendremos clases con muchos métodos, difíciles de comprender y de mantener. Además, estas funcionalidades darán lugar a métodos (**calculaCosteEmpresa** y **envíaEmailComercial**) sin relación entre ellos y sin relación entre el núcleo de los objetos con la diferencia, por ejemplo, del método **agregaFilial** que contribuye a componer los objetos.

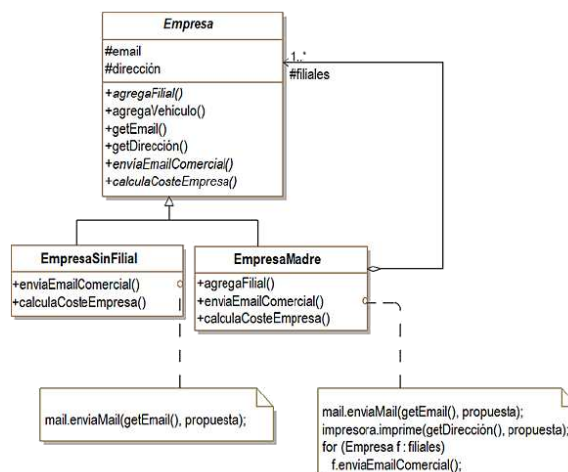


Figura 10: Múltiples funcionalidades en el seno de objetos compuestos

El patrón Visitor permite implementar las nuevas funcionalidades en un objeto separado llamado visitante. Cada visitante establece una funcionalidad para varias clases incluyendo para cada una de ellas un método de implementación llamado visita y cuyo parámetro está tipado según la clase a visitar.

A continuación, el visitante se transmite al método **aceptaVisitante** de estas clases. Este método invoca al método del visitante correspondiente a su clase. Sea cual sea el número de funcionalidades a implementar en un conjunto de clases, sólo debe escribirse el método **aceptaVisitante**. Puede ser necesario ofrecer la posibilidad al visitante de acceder a la estructura interna del objeto visitado (preferentemente mediante accesos en modo lectura como en este caso los métodos **getNombre**, **getEmail** y **getDirección**).

Si los objetos son compuestos entonces su método **aceptaVisitante** llama al método **aceptaVisitante** de sus componentes. Es el caso aquí para cada instancia de la clase **EmpresaMadre** que llama al método **aceptaVisitante** de sus filiales.

El diagrama de clases de la Figura 11 ilustra la implementación del patrón Visitor. La interfaz Visitante incluye la firma de los métodos que implementan la funcionalidad para cada clase a visitar. Esta interfaz posee dos subclases de implementación, una por cada funcionalidad.

1.6.3. Estructura

La figura 12 detalla la estructura genérica del patrón.

1.6.4. Participantes

Los participantes del patrón son los siguientes:

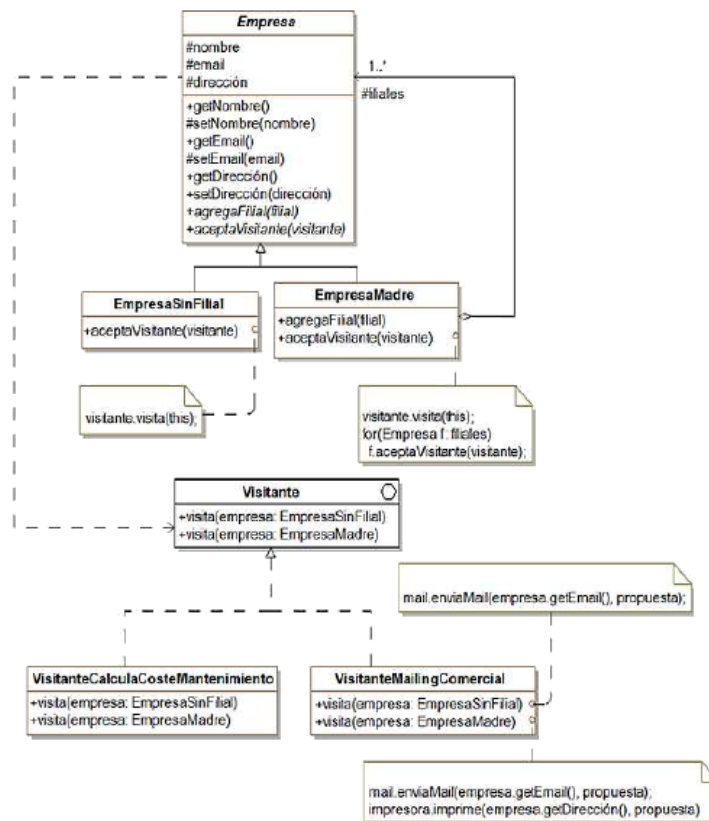


Figura 11: Aplicación del patrón Visitor para agregar una funcionalidad de mailing y el cálculo del coste de mantenimiento

- **Visitante** es la interfaz que incluye la firma de los métodos que realizan una funcionalidad en un conjunto de clases. Existe un método para cada clase que recibe como argumento una instancia de esta clase.
- **VisitanteConcreto1** y **VisitanteConcreto2** ((**VisitanteCálculoCosteEmpresa** y **VisitanteMailingComercial**) implementan los métodos que realizan la funcionalidad correspondiente a la clase.
- **Elemento** (**Empresa**) es una clase abstracta súperclase de las clases de elementos. Incluye el método abstracto **aceptaVisitante** que acepta un visitante como argumento.
- **ElementoConcreto1** y **ElementoConcreto2** (**EmpresaSinFilial** y **EmpresaMadre**) implementa el método **aceptaVisitante** que consiste en volver a llamar al visitante a través del método correspondiente de la clase.

1.6.5. Colaboraciones

Un cliente que utiliza un visitante debe en primer lugar crearlo como instancia de la clase de su elección y, a continuación, pasarlo como argumento al método **aceptaVisitante** de un conjunto de elementos.

El elemento vuelve a llamar al método del visitante que corresponde con su clase. Le pasa una referencia hacia sí mismo como argumento para que el visitante pueda acceder a su estructura interna.

1.6.6. Dominios de Aplicación

El patrón se utiliza en los casos siguientes:

- Es necesario agregar numerosas funcionalidades a un conjunto de clases sin volverlas pesadas.
- Un conjunto de clases poseen una estructura fija y es necesario agregarles funcionalidades sin modificar su interfaz.

Si la estructura del conjunto de clases a las que es necesario agregar funcionalidades cambia a menudo, el patrón Visitor no se adapta bien. En efecto, cualquier modificación de la estructura implica una modificación de cada visitante, lo cual puede tener un costo elevado.

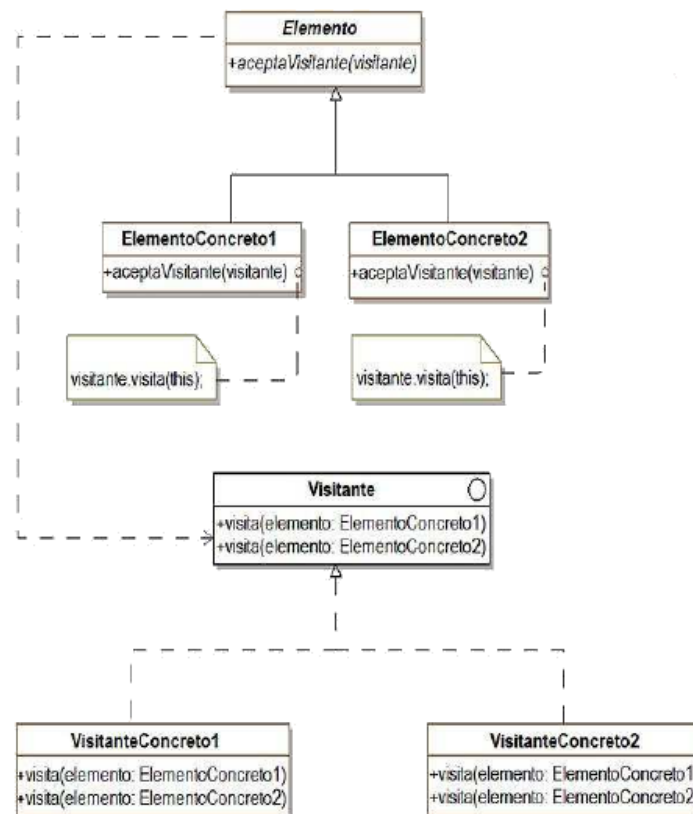


Figura 12: Estructura del patrón Visitor

1.6.7. Ejemplo en Java

```

1 public abstract class Empresa {
2
3     protected String nombre, email, direccion;
4     public Empresa(String nombre, String email, String direccion) {
5         this.setNombre(nombre);
6         this.setEmail(email);
7         this.setDireccion(direccion);
8     }
9
10    public String getNombre() {
11        return nombre;
12    }
13
14    protected void setNombre(String nombre) {
15        this.nombre = nombre;
16    }
17
18    public String getEmail() {
19        return email;
20    }
21
22    protected void setEmail(String email) {
23        this.email = email;
24    }
25
26    public String getDireccion() {
27        return direccion;
28    }
29
30    protected void setDireccion(String direccion) {
31        this.direccion = direccion;
32    }
33
34    public abstract boolean agregaFilial(Empresa filial);
35    public abstract void aceptaVisitante(Visitante visitante);
36 }

```

El código fuente de la subclase `EmpresaSinFilial` aparece a continuación. El método `aceptaVisitante` vuelve a llamar al método `visita` del visitante.

```
1 public class EmpresaSinFilial extends Empresa {
2
3     public EmpresaSinFilial(String nombre, String email, String direccion) {
4         super(nombre, email, direccion);
5     }
6
7     public void aceptaVisitante(Visitante visitante) {
8         visitante.visita(this);
9     }
10
11     public boolean agregaFilial(Empresa filial) {
12         return false;
13     }
14 }
```

El código fuente de la subclase `EmpresaMadre` aparece a continuación. El método `aceptaVisitante` vuelve a llamar al método `visita` del visitante y, a continuación, invoca al método `aceptaVisitante` de sus filiales.

```
1 import java.util.ArrayList;
2 import java.util.List;
3 public class EmpresaMadre extends Empresa {
4
5     protected List<Empresa> filiales = new ArrayList<Empresa>();
6     ...
7
8     public EmpresaMadre(String nombre, String email, String direccion) {
9         super(nombre, email, direccion)
10    }
11    ...
12
13    public void aceptaVisitante(Visitante visitante) {
14        visitante.visita(this);
15        for (Empresa filial: filiales)
16            filial.aceptaVisitante(visitante);
17    }
18
19    public boolean, agregaFilial(Empresa filial) {
20        return filiales.add(filial);
21    }
22 }
```

La interfaz `Visitante` incluye la firma de los dos métodos, uno por cada clase que tenga que ser visitada.

```
1 public interface Visitante {
2     void visita(EmpresaSinFilial empresa);
3     void visita(EmpresaMadre empresa);
4 }
```

La clase `VisitanteMailingComercial` envía los correos electrónicos a las empresas implementando la interfaz `Visitante`. Las empresas que poseen filiales reciben una propuesta particular y además por correo postal. Aquí se simula mediante impresiones por pantalla.

La clase `VisitanteCalculaCosteMantenimiento` no se ha tenido en cuenta en este ejemplo.

```
1 public class VisitanteMailingComercial implements Visitante {
2
3     public void visita(EmpresaSinFilial empresa) {
4         System.out.println("Envia un email a " + empresa.getNombre() + " direccion: " +
5             empresa.getEmail() + " Propuesta comercial para su empresa");
6     }
7
8     public void visita(EmpresaMadre empresa) {
9
10        System.out.println("Envia un email a " + empresa.getNombre() + " direccion: " +
11            empresa.getEmail() + " Propuesta comercial para su grupo");
12        System.out.println("Impresion de un correo para " + empresa.getNombre() + " direccion: " +
13            empresa.getDireccion() +
14            " Propuesta comercial para su grupo");
15    }
16 }
```

Por último la clase `Usuario` crea un grupo (grupo 2) constituido por la empresa 3 y por el grupo 1, el cual está constituido por la empresa 1 y la empresa 2.

A continuación procede a enviar los correos electrónicos (mailing) a todas las empresas del grupo 2 invocando a su método `aceptaVisitante` con un visitante, instancia de la clase `VisitanteMailingComercial`.

```
1 public class Usuario {
2
3     public static void main(String[] args) {
4
5         Empresa empresa1 = new EmpresaSinFilial("empresa1", "info@empresa1.com", "calle de la empresa 1");
6         Empresa empresa2 = new EmpresaSinFilial("empresa2", "info@empresa2.com", "calle de la empresa 2");
7         Empresa grupo1 = new EmpresaMadre("grupo1", "info@grupo1.com", "calle del grupo 1");
8         grupo1.agregaFilial(empresa1);
9         grupo1.agregaFilial(empresa2);
10        Empresa empresa3 = new EmpresaSinFilial("empresa3", "info@empresa3.com", "calle de la empresa 3");
11        Empresa grupo2 = new EmpresaMadre("grupo2", "info@grupo2.com", "calle del grupo 2");
12        grupo2.agregaFilial(grupo1);
13        grupo2.agregaFilial(empresa3);
14        grupo2.aceptaVisitante(new VisitanteMailingComercial());
15    }
16 }
```

El resultado de la ejecución es el siguiente.

```
Envía un email a grupo2 dirección: info@grupo2.com
Propuesta comercial para su grupo
Impresión de un correo para grupo2 dirección: calle del grupo 2
Propuesta comercial para su grupo
Envía un email a grupo1 dirección: info@grupo1.com
Propuesta comercial para su grupo
Impresión de un correo para grupo1 dirección: calle del grupo 1
Propuesta comercial para su grupo
Envía un email a empresa1 dirección: info@empresa1.com
Propuesta comercial para su empresa
Envía un email a empresa2 dirección: info@empresa2.com
Propuesta comercial para su empresa
Envía un email a empresa3 dirección: info@empresa3.com
Propuesta comercial para su empresa
```